

GitHub F1 CSV API - Line-by-Line Explanation

This guide explains how to use the GitHub API to discover CSV files in the toUpperCase78/formula1-datasets repository and convert the repository's folder structure into a nested Python dictionary.

1. Complete Code

```
import requests

OWNER = "toUpperCase78"
REPO = "formula1-datasets"
BRANCH = "master"

url = f"https://api.github.com/repos/{OWNER}/{REPO}/git/trees/{BRANCH}?recursive=1"

response = requests.get(url)
response.raise_for_status()

tree = response.json()["tree"]

def build_structure(tree):
    root = {}

    for item in tree:
        if item["type"] != "blob":
            continue

        if not item["path"].lower().endswith(".csv"):
            continue

        parts = item["path"].split("/")

        current = root

        if len(parts) == 1:
            current.setdefault("main", []).append(parts[0])
            continue

        for folder in parts[:-1]:
            if folder not in current:
                current[folder] = {}

            current = current[folder]

        current.setdefault("_files", []).append(parts[-1])

    return root

structure = build_structure(tree)

print(structure)
```

2. Import requests

```
import requests
```

What it does: Imports the requests library, which lets Python send HTTP requests to websites and APIs. We use it to communicate with GitHub.

3. Repository Information

```
OWNER = "toUpperCase78"
REPO = "formula1-datasets"
BRANCH = "master"
```

OWNER is the GitHub username that owns the repository. REPO is the repository name. BRANCH is the branch whose files we want. Together these identify toUpperCase78/formula1-datasets on the master branch.

4. Build the GitHub API URL

```
url = f"https://api.github.com/repos/{OWNER}/{REPO}/git/trees/{BRANCH}?recursive=1"
```

The f creates an f-string, allowing variables inside curly braces. The final URL becomes the GitHub Git Trees API endpoint. recursive=1 tells GitHub to include files inside subfolders, not just the repository root.

Conceptually, the resulting URL is:

```
https://api.github.com/repos/toUpperCase78/formula1-datasets/git/trees/master?recursive=1
```

5. Send the Request

```
response = requests.get(url)
```

This sends an HTTP GET request to GitHub. The returned response object contains the HTTP status, headers, and the JSON data returned by GitHub.

6. Check for Errors

```
response.raise_for_status()
```

This raises a Python exception if GitHub returns an HTTP error such as 404 or 403. If the request succeeds, execution continues normally.

7. Extract the Repository Tree

```
tree = response.json()["tree"]
```

response.json() converts GitHub's JSON response into normal Python objects (dictionaries and lists). ["tree"] selects the list containing repository files and folders.

Example conceptually:

```
{
  "tree": [
    {"path": "drivers.csv", "type": "blob"},
    {"path": "PreviousSeasons", "type": "tree"},
    {"path": "PreviousSeasons/2022/drivers.csv", "type": "blob"}
  ]
}
```

8. Create build_structure()

```
def build_structure(tree):
```

Defines a function named build_structure. It receives GitHub's tree list and will return the nested dictionary we want.

9. Create the Root Dictionary

```
root = {}
```

Creates an empty dictionary. This dictionary will gradually become the complete representation of the repository.

10. Loop Through Every GitHub Item

```
for item in tree:
```

Processes every file or folder returned by GitHub, one item at a time.

11. Keep Only Files

```
if item["type"] != "blob":  
    continue
```

GitHub represents a file as a blob and a directory as a tree. This condition skips directories themselves because the file paths already tell us where CSVs are located.

12. Keep Only CSV Files

```
if not item["path"].lower().endswith(".csv"):  
    continue
```

item["path"] gives the file path. lower() makes the check case-insensitive. endswith(".csv") checks whether the path ends with .csv. The not means non-CSV files are skipped.

13. Split the File Path

```
parts = item["path"].split("/")
```

Splits a path at each slash. For example:

```
PreviousSeasons/2022/drivers.csv
```

becomes:

```
["PreviousSeasons", "2022", "drivers.csv"]
```

14. Start at the Root

```
current = root
```

The variable current points to the dictionary location where we are currently adding folders or files. As we move through nested folders, current is changed to point deeper into the dictionary.

15. Handle CSVs in the Main Folder

```
if len(parts) == 1:  
    current.setdefault("main", []).append(parts[0])  
    continue
```

If the path contains only one part, the CSV is directly in the repository root. For example, drivers.csv becomes ["drivers.csv"]. setdefault("main", []) creates a main list if it does not exist, then append() adds the filename. continue moves to the next GitHub item.

16. Process Nested Folders

```
for folder in parts[:-1]:
```

parts[:-1] means all parts except the last one. Since the last part is the filename, this loop processes only the folders. For ["PreviousSeasons", "2022", "drivers.csv"], the folders are PreviousSeasons and 2022.

17. Create a Folder If Needed

```
if folder not in current:  
    current[folder] = {}
```

Checks whether the current folder already exists in the dictionary. If not, creates it as an empty dictionary.

18. Move Deeper into the Dictionary

```
current = current[folder]
```

Changes current so that it points to the dictionary for the folder we just entered. This is how the code follows the GitHub folder hierarchy.

19. Add the CSV to Its Folder

```
current.setdefault("_files", []).append(parts[-1])
```

parts[-1] means the last element, which is the CSV filename. setdefault("_files", []) creates a list for files in the current folder if needed, then append() adds the filename.

20. Why Use _files?

A folder can contain both CSV files and subfolders. Using a special key such as _files lets the dictionary represent both without losing information.

```
{
  "PreviousSeasons": {
    "2022": {
      "_files": [
        "drivers.csv",
        "races.csv"
      ],
      "Sprint": {
        "_files": [
          "results.csv"
        ]
      }
    }
  }
}
```

21. Return the Finished Dictionary

```
return root
```

Returns the completed nested dictionary from the function.

22. Run the Function

```
structure = build_structure(tree)
print(structure)
```

The function receives the GitHub tree and produces the nested dictionary. The result is stored in structure, and print() displays it.

23. How the Transformation Works

GitHub path:
PreviousSeasons/2022/drivers.csv

```
|
| split("/")
v
```

```
["PreviousSeasons", "2022", "drivers.csv"]
```

```
|
| build folders
v
```

```
{
  "PreviousSeasons": {
    "2022": {
      "_files": ["drivers.csv"]
    }
  }
}
```

```

    }
  }
}

```

24. Important Improvement for Your F1 Project

If the ultimate goal is to load the CSVs directly into pandas, a useful next step is to store each filename together with its GitHub raw download URL. Then your dictionary can become a direct data source for `pd.read_csv()`, without manually downloading files.

```

{
  "main": {
    "drivers.csv": "https://raw.githubusercontent.com/..."
  },
  "PreviousSeasons": {
    "2022": {
      "drivers.csv": "https://raw.githubusercontent.com/..."
    }
  }
}

```

Key idea: GitHub API -> repository tree -> file paths -> split paths -> nested dictionary -> pandas.